

One Timeline, Many Renderings: A Wolfram Language Paclet for Heterogeneous Musical Output

AuthorA¹, AuthorB², and AuthorC³ *

¹ InstituteA

² InstituteB

³ InstituteC

Abstract. Algorithmic composition rarely ends in a single medium: the same musical plan may need to become a Csound score for synthesis, engraved notation for performers, a real-time control stream, and a click track for rehearsal. When each artifact is authored or repaired separately, their timelines drift apart. This paper presents Temporal System, a Wolfram Language paclet that treats heterogeneous output as a projection problem: one immutable store of typed musical entities, placed on a rational beat timeline, is compiled by per-backend renderers into Csound, MusicXML, and OSC artifacts that stay mutually synchronous because they derive from one store: Csound carries the electroacoustic and synthesis layer, MusicXML the symbolic score (in beta against a reference schema), and OSC the real-time layer for live electronics. The paper describes the problem and the design decisions behind the paclet, its anatomy (temporal, semantic, and rendering layers across fourteen modules), and the implemented renderers. Particular attention goes to the Csound renderer, the backend that fixes the paclet’s rendering idiom: compiled tempo maps produce onset and duration seconds only at render time, session tuning produces p4 frequencies, curve entities compile to k-rate control signals bound to score p-fields, and orchestras remain ordinary `.orc` files addressed by stable named instruments, continuing the score–orchestra separation that Csound inherits from the MUSIC N family. A dedicated click-track backend derives rehearsal audio from the same meter and tempo data and reuses the Csound serializer, showing how inexpensively a further projection can be added within a rendering family once the temporal core is shared. Figures include a timeline visualization exported directly from the paclet.

Keywords: Csound, algorithmic composition, music representation, Wolfram Language, click track

1 Introduction

Long before rendering became plural, computing composers had already separated structure from sound. At Bell Labs, James Tenney generated note lists for the MUSIC N programs to play [2,3]; in Utrecht, Gottfried Michael Koenig’s *Project 1* and *Project 2* computed a piece as tables whose realization belonged to a later stage—players, the studio and, in his last electronic works, Csound itself [1]. In both practices the program owns *time and structure* while a separate system owns *sound*: the division Csound institutionalizes as the orchestra–score split [4].

Contemporary practice multiplies the realization stages. A mixed work may need a `.csd` for synthesis, MusicXML for engraving, a timed control stream for the live-electronics layer, and a click track for the performers. These formats encode time differently—seconds, divisions per quarter, milliseconds, audio samples—so maintaining them by hand means maintaining four diverging timelines. Computer-assisted composition environments have long addressed parts of this problem [5,6]; Temporal System, the Wolfram Language paclet described here [14], addresses it by making one authored timeline the only source of truth and demoting every output file to a derived artifact.

The idea of one model feeding many outputs is itself well established. *music21* exposes a single object model exporting to MusicXML, MIDI, Lilypond and text [7]; *abjad* drives Lilypond engraving from a persistent Python score model [8]; MEI encodes a canonical score that Verovio projects to many targets [9,10]; and *bach* and *cage* carry a score/timeline abstraction into Max for real-time realization [11]. These systems are organized around notation or engraving, or—where realization is

* Please place acknowledgement here.

real-time—around a single host environment; in each, time is ultimately notational or measured in seconds. Temporal System makes something else primary: one beat-space infrastructure, in which onsets are exact Rationals—infinite in precision and musical rather than clock time—drives every layer at once, the fixed electroacoustic through Csound, the symbolic score through MusicXML, and the live stream through OSC, with conversion to seconds, hertz and samples deferred to each renderer. What makes this practical is the host language: the store, its exact tempo-map inversion, its session-parameterized tuning and its restrictive per-backend contracts are all expressed in Wolfram Language, a high-level symbolic system in which rationals stay exact, transformations are functional and composable, and a vast built-in library—symbolic algebra, signal processing, visualization, the packet’s own timeline graphics included—is available in the same environment where the timeline is authored. Authoring the piece and compiling its every projection therefore share one language, not a chain of separate programs.

Temporal System’s design decisions are few and strict: (i) events live in an immutable store of typed entities whose onsets are exact Rationals in beat space, never seconds; (ii) meter is authored state, never inferred from durations; (iii) reference tuning is a session parameter, never a constant in conversion code; (iv) the packet generates *scores only*—synthesis instruments remain external `.orc` files, as in Koenig’s and Tenney’s workflows; (v) the shared store is permissive, while each renderer enforces a restrictive contract for its own payload.

One trade-off is worth stating plainly. Authoring the source of truth requires Wolfram Language, a commercial environment, whereas Csound and much of its ecosystem are open-source and cost-sensitive. The dependency is confined to authoring: the artifacts the packet emits—the `.csd` with its `#included` `.orc` files, MusicXML, and the OSC JSON—are plain text in open formats and need no Wolfram runtime to compile, engrave, or dispatch. A freely available Wolfram Engine covers development, evaluation, and non-commercial use, though commercial or organizational production still requires a paid license.

2 Anatomy of the Packet

The packet comprises fourteen modules exposing 80 public functions, organized in the three layers of Fig. 1: **core** (entities, store, tempo, session), **rendering** (dispatcher and four backends), and **util** (pitch, rhythm, dynamics, validation, timeline graphics). All functions are pure; store operations return new stores.

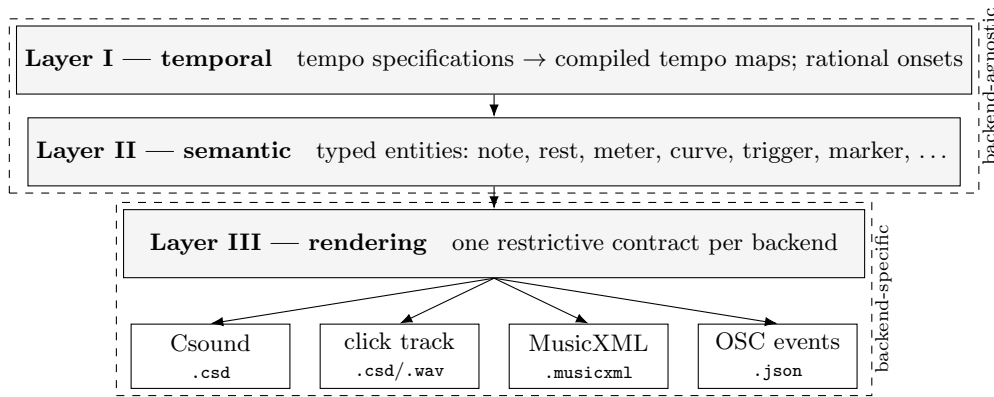


Fig. 1. The three-layer model. Adding a backend adds a contract in Layer III; Layers I and II never change to accommodate an output format.

Layer I. Tempo is authored as a *tempoSpec*, an ordered list of {beat, BPM} points, and compiled to a piecewise-linear *tempoMap* in {beat, seconds} form with exact analytical inversion. Onsets and durations are Rationals in musical duration space; conversion to seconds or samples happens only inside renderers.

Layer II. The store groups entities by type (`stemData[type][id]`), giving $O(1)$ typed access across eleven types: note, rest, trigger, curve, process, state, marker, interval, trajectory, annotation,

gesture. A note carries pitch (spelling plus cents, MIDI float, or frequency), duration, dynamic, and a **rendering** association keyed by backend id—the one place where backend-specific payloads travel with the entity. Meter is a state entity: measure structure is authored, not guessed. A session configuration carries everything a renderer may not hardcode, starting with **tuningA4** (default 440 Hz, freely reassignable).

Layer III. Constructors validate only the shared shape and require each **rendering[backend]** payload to be an association; the keys inside belong to the backend contract. Malformed Csound payloads are therefore Csound contract errors, local to one renderer, and each backend declares a fallback policy (warn, approximate) for material it cannot express.

The packet also inspects itself: **drawTimeline** renders any **stemData** as lane-based graphics with the metric grid derived from the authored meter entities. Figure 2 is such an export—the most complex fixture of the packet’s Csound test suite, with two instruments, six control curves, four triggers and four meter signatures—produced by a single function call, with no external tool. Every fixture in this suite renders to a **.csd** that Csound compiles without error, and its rendered audio was auditioned.

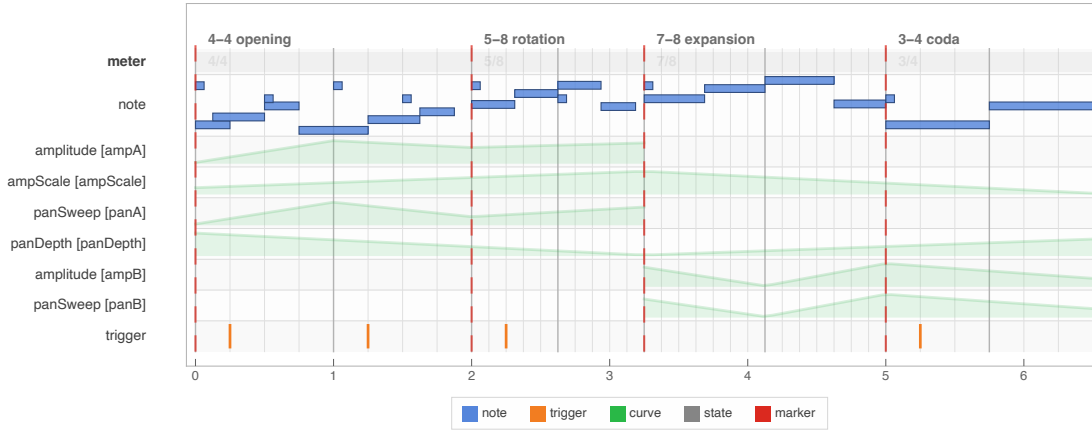


Fig. 2. A timeline exported directly by the packet (**drawTimeline**): the most complex fixture of the Csound test suite—sixteen **sine_osc** notes (several with cent offsets) and eight **click** hits, four triggers, six overlapping curves on amplitude and pan parameters, labeled section markers, and a $4/4 \rightarrow 5/8 \rightarrow 7/8 \rightarrow 3/4$ meter grid. It is a stress fixture that exercises many entity types at once, not a representative composition.

3 The Implemented Renderers

A backend is data: an association with an id, a renderer, a serializer, the primitive types it emits, and its fallback policy. The generic dispatcher **renderTo[stemData, tempo, backend, config, registry]** is the single entry point for all outputs. Four backends exist. *Csound* compiles the whole **stemData** to score and orchestra primitives (Sect. 4). *Click* derives a standalone rehearsal track from meter and tempo alone (Sect. 5). *MusicXML* builds measures from meter state, splits notes at bar-lines into tied fragments, and validates measure capacity, part coverage, and tie and chord integrity; it is explicitly a lossy projection with a declared coverage profile [12], currently in beta, its export checked by round-tripping through a commercial engraving application and comparing the re-import against the source **stemData** for tie, meter and part-separation fidelity. *OSC* emits a schedule-ready JSON event list in milliseconds—curves sampled at a configurable control rate—for dispatch as OSC messages [13] to whatever is listening: a graphical patcher as readily as a running Csound instance through its OSC opcodes. Not every entity exists everywhere: curves become k-rate control signals in the Csound projection but have no notation image; spelling matters to MusicXML and not to synthesis. The packet records these fidelity differences per entity-backend pair instead of pretending projections are lossless.

4 The Csound Renderer

Csound is the oldest backend and the one that fixes the packet’s rendering idiom, carrying the electroacoustic and synthesis output (Fig. 3). Its p-field convention is stable: **p1** instrument, **p2** onset seconds, **p3** duration seconds, **p4** frequency in Hz, **p5** amplitude in $[0, 1]$, **p6+** passed through from the entity’s `rendering["csound"]` payload. The renderer takes the stemData as a whole (`renderCSnd`): notes and triggers become **i** statements, markers become onset-ordered score comments, and each curve becomes a k-rate control signal held in a global variable—an **f**-table reader or a generated **transeg**, per session choice. In the **f**-table form the curve is stored as a GEN-2 table—512 points by default, itself a session parameter—read at k-rate by `tablei`, which interpolates linearly between samples.

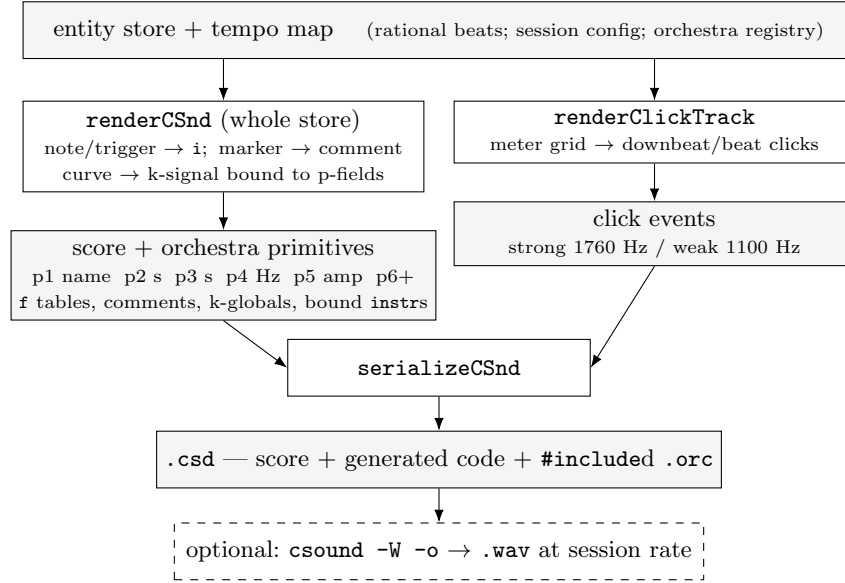


Fig. 3. Both Csound-family renderers share one serializer; source instruments stay in external `.orc` files, and curve bindings add generated copies without touching them.

Two decisions matter most. First, *late conversion*: because onsets stay Rational until this point, **p2/p3** are computed from the compiled tempo map, **p4** from spelling and cents under the session’s `tuningA4`, and **p5** from the dynamic marking, all at render time. Retuning a session to 432 Hz or editing its tempo specification regenerates every second and every frequency without touching one entity. Second, *named instruments*: an orchestra registry is loaded from a directory of `.orc` files, the filename being the stable id used both in `instr` definitions and in entity payloads. No numeric renumbering ever occurs, so the note

```
i "sine_osc" 0. 4. 440. 0.65 0.5
```

(A4, *mf*, two whole notes at 120 BPM, actual packet output) remains readable and stable as the orchestra grows.

Curves reach the sounding notes through declared bindings: by default an `amplitude` curve multiplies **p5**. For each affected instrument the renderer copies the whole `instr` body from its source `.orc` into a *bound copy* and rewrites only the use-sites of the bound p-fields, so the control-signal globals multiply or replace them there; everything else—the `pcount()` guards, the envelope generation, the `limit()` clamps, the oscillator—is reproduced verbatim. The affected notes are re-pointed to this copy, and several curves may compose on one p-field: in Fig. 2 two amplitude curves and an `ampScale` curve multiply into **p5**, while `panSweep` replaces and `panDepth` multiplies **p6**. The bound copy is regenerated from the current `.orc` at every render and the source orchestra is never edited, so the copy cannot drift from its source; the binding relies on the packet’s p-field convention to locate those use-sites. The serializer writes the globals and the generated instruments into the orchestra

beside the registry's `#include` lines, sorts score statements by onset, and can still invoke the Csound binary to compile the exported `.csd` to audio.

The same amplitude curve lives very differently outside Csound. Projected to the OSC backend it is not bound into an instrument but sampled at the session control rate into timed messages: a linear ramp over two seconds at 10 Hz becomes the stream below (actual packet output, excerpt of `example/paper_curve_osc.json`):

```
{ "version": 1, "sampleRate": 44100, "duration_ms": 1900,
  "events": [
    { "time_ms": 0, "address": "/param/amplitude", "args": [0.0] },
    { "time_ms": 100, "address": "/param/amplitude", "args": [0.05] },
    { "time_ms": 200, "address": "/param/amplitude", "args": [0.1] },
    ...
    { "time_ms": 1900, "address": "/param/amplitude", "args": [0.95] } ] }
```

The beat-space curve and the tempo map are the same as in the Csound projection; one binds the curve to a score p-field, the other streams it as OSC control messages in milliseconds. It is a genuinely non-Csound artifact—the heterogeneity the architecture exists to make cheap.

5 A Click Track from the Same Timeline

Click tracks for mixed music are usually rebuilt by hand in a DAW whenever the score changes—precisely the divergence this architecture exists to remove. The click backend treats rehearsal audio as one more projection: it reads only the authored meter entities and the tempo map, builds the measure grid, and emits one short `i` event per beat, accenting each downbeat. Every parameter is session configuration—beat unit (1/4), click duration (1/16), strong/weak frequencies (1760/1100 Hz), amplitudes (0.9/0.45), and pan—and the click instrument itself is an ordinary three-line `.orc` file. Because the renderer walks the same meter grid and tempo map as every other backend, meter and tempo changes are followed with no additional authoring. The output below is actual packet output for two 4/4 bars at 120 BPM (downbeats at 0 and 2 s):

```
<CsScore>
f 1 0 16384 10 1
i "click" 0. 0.125 1760. 0.9 0.5
i "click" 0.5 0.125 1100. 0.45 0.5
i "click" 1. 0.125 1100. 0.45 0.5
i "click" 1.5 0.125 1100. 0.45 0.5
i "click" 2. 0.125 1760. 0.9 0.5
...
```

The backend reuses `serializeCSnd` unchanged (Fig. 3), inheriting `#include` handling and optional compilation to `.wav` for distribution to performers. Implementing it required no change to Layers I–II. Because the click track is itself a Csound-family output, it shows how cheap a further projection becomes once the temporal core is shared—here within one serializer family; the genuinely different time encodings, notation's divisions and OSC's milliseconds, are the province of the MusicXML and OSC contracts.

6 Conclusions

Temporal System contributes an architecture, not a synthesis method: an immutable rational-beat store from which Csound scores, click tracks, notation, and real-time event streams are compiled as unequal but coordinated projections. Csound currently plays the part of reference renderer because it makes the contract concrete—late conversion of time, pitch, and dynamics; stable named instruments; orchestras kept as Csound code. Current work includes stabilizing the MusicXML backend beyond beta and a hierarchical generation layer above the store. Sixty years after *Project 1* and the Bell Labs note-lists, separating computed time from rendered sound still pays: it is what lets one timeline become many renderings.

References

1. Koenig, G.M.: Project Two: A Programme for Musical Composition. *Electronic Music Reports* 3, 4–161. Institute of Sonology, Utrecht (1970)
2. Tenney, J.: Computer Music Experiences, 1961–1964. *Electronic Music Reports* 1(1), 23–60. Institute of Sonology, Utrecht (1969)
3. Mathews, M.V.: *The Technology of Computer Music*. MIT Press, Cambridge (1969)
4. Lazzarini, V., Yi, S., ffitch, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: *Csound: A Sound and Music Computing System*. Springer (2016)
5. Assayag, G., Rueda, C., Laurson, M., Agon, C., Delerue, O.: Computer-Assisted Composition at IRCAM: From PatchWork to OpenMusic. *Computer Music Journal* 23(3), 59–72 (1999)
6. Taube, H.: *Notes from the Metalevel: Introduction to Algorithmic Music Composition*. Taylor & Francis, London (2004)
7. Cuthbert, M.S., Ariza, C.: music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data. In: *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 637–642. Utrecht (2010)
8. Bača, T., Oberholtzer, J.W., Treviño, J., Adán, V.: Abjad: An Open-Source Software System for Formalized Score Control. In: *Proceedings of the First International Conference on Technologies for Music Notation and Representation (TENOR)*, pp. 162–169. Paris (2015)
9. Hankinson, A., Roland, P., Fujinaga, I.: The Music Encoding Initiative as a Document-Encoding Framework. In: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 293–298. Miami (2011)
10. Pugin, L., Zitellini, R., Roland, P.: Verovio: A Library for Engraving MEI Music Notation into SVG. In: *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 107–112. Taipei (2014)
11. Agostini, A., Ghisi, D.: A Max Library for Musical Notation and Computer-Aided Composition. *Computer Music Journal* 39(2), 11–27 (2015)
12. Good, M.: MusicXML for Notation and Analysis. In: Hewlett, W.B., Selfridge-Field, E. (eds.) *The Virtual Score: Representation, Retrieval, Restoration*, pp. 113–124. MIT Press, Cambridge (2001)
13. Wright, M., Freed, A.: Open SoundControl: A New Protocol for Communicating with Sound Synthesizers. In: *Proceedings of the International Computer Music Conference*, pp. 101–104. Thessaloniki (1997)
14. Wolfram Research: Wolfram Language and Paclet Documentation. <https://reference.wolfram.com/language/>